

自我介绍

はじめまして、徐耀晨と申します。杭州電子科技大学のソフトウェアエンジニアリング学科を卒業いたしました。

大学ではソフトウェアエンジニアリングを専攻し、特に Java の基礎知識と開発スキルを学びました。菜鳥ネットワークテクノロジー有限公司で Java 開発のインターンを務めた際、物流料金計上関連の開発業務を担当し、Git を活用した複数人協働の開発フローを熟知し、Git の基本的な操作を習得しました。

今後は御社でソフトウェア開発に携わりながら、自分の強みである協調性と Git のスキルを生かし、チームの成果に貢献できる人材へ成長していきたいと考えております。どうぞよろしくお願いいたします。

はじめまして、徐耀晨（じょ ようしん）と申（もう）します。杭州電子科技大学（こうしゅうでんしかがくだいがく）のソフトウェアエンジニアリング学科（がっか）を卒業（そつぎょう）いたしました。

大学（だいがく）ではソフトウェアエンジニアリングを専攻（せんこう）し、特（とく）に Java の基礎知識（きそちしき）と開発（かいはつ）スキルを学（まな）びました。

菜鳥（さいにゃく）ネットワークテクノロジー有限公司（さいにゃくネットワークテクノロジーゆうげんこうし）で Java 開発（かいはつ）のインターンを務（つと）めた際（さい）、物流（ぶつりゅう）料金計上（りょうきんけいじょう）関連（かんれん）の開発（かいはつ）業務（ぎょうむ）を担当（たんとう）し、Git を活用（かつよう）した複数人（ふくすうにん）協働（きょうどう）の開発フローを熟知（じゅくち）し、Git の基本的（きほんてき）な操作（そうさ）を習得（しゅうとく）しました。

今後（こんご）は御社（おんしゃ）でソフトウェア開発（かいはつ）に携（たずさ）わりながら、自分（じぶん）の強み（つよみ）である協調性（きょうちょうせい）と Git のスキルを生（い）かし、チームの成果（せい）かに貢献（こうけん）できる人材（じんざい）へ成長（せいちょう）していきたいと考（かんが）えております。どうぞよろしくお願いいたします（ねが）。

志愿动机

御社を志望した理由は、御社が物流システム分野に豊富な実績とノウハウを有し、物流業界の課題解決に積極的に取り組んでいる点に強く魅力を感じたからです。

また、御社のホームページで入出荷管理システムの取り組みを拝見し、自身が物流料金計上関連の Java 開発実務経験を積んでいることから、御社の物流システム開発に Java 開発力とチーム協働経験を活かせると確信しており、一員となって御社の事業発展に貢献したいと強く願っております。

御社を志望した理由は、御社が物流システム分野に豊富(ほうふ)な実績とノウハウを有(ゆう)し、物流業界の課題解決(かだいかいけつ)に積極的に取り組んでいる点に強く魅力(みりょく)を感じたからです。

また、御社のホームページで入出荷管理システムの取り組みを拝見し、自身(じしん)が物流(ぶつりゅう)料金計上(りょうきんけいじょう)関連(かんれん)のJava開発(かいはつ)実務(じつむ)経験を積んでいることから、御社の物流システム開発にJava開発力とチーム協働(きょうどう)経験を活(い)かせると確信しており、一員となって御社の事業発展に貢献したいと強く願っております。

自我PR

私の強みは、問題解決力と協調性です。

例えば、開発実習のプロジェクトに参加した際、先輩とメタデータ加工クラスを実装中にメソッド名の重複によるコードの上書きという課題に直面しました。

その時、私は慌てず状況を整理し、Git で新しいブランチを作成し、クラス名や実装を調整することでコンフリ

クトを回避するよう自発的に対応しました。

その結果、チームの開発進行に影響を与えず問題を解決でき、先輩からも冷静に状況に対応できると評価されました。

この経験から培った問題解決力と協調性を活かし、御社でもチーム開発に貢献し、安定的に成果を出していきたいと考えております。

協調性（きょうちょうせい）

重複（じゅうふく）

上書（うわが）き

回避（かいひ）

弱点

短所は、開発経験がまだ少ないことです。ただ、その分、学習能力には自信があります。大学時代はいくつかの開発プロジェクトに取り組みましたが、実務レベルでは、まだ初心者の段階だと思っています。だからこそ、新しい環境に飛び込んで、一つ一つ吸収しながら、着実に成長していきたいと考えています。現在も、Nettyの勉強を続けており、できることから積極的に挑戦しています

協調性（きょうちょうせい）

重複（じゅうふく）

上書（うわが）き

回避（かいひ）

取（と）り組（く）： 致力于、投身于、专心做

飛び込む（とびこむ）： 跳入、主动进入、投身进入

着実（ちゃくじつ）

挑戦（ちょうせん）

反問

1 御社様は、例えば、現在開発中のプロジェクトでは、新機能のリリースサイクルはどのくらいの頻度ですか。

（目前新功能的发布周期大概是怎样的频率在运行呢？）

💡 为什么这个问题能拿高分？

展现敏捷思维（Agile）： 你关注的是“发布频率（Release Cycle）”，这听起来非常专业（DevOps/CI/CD视角）

1 プロジェクトがない待機期間の給与は、どのように計算されますか？

ぷろじえくとがないたいききかんの きゅうよ は、どのように けいさん されますか？ ）

2 貴社では厚生年金と社会保険の加入手続きをしていただけますか。

（きしゃ では こうせいねんきん と しゃかいほけん の かにゆう てつづき を していただけますか。）

本日は貴重なお話をありがとうございました。

遇到的困难

これまでの人生の中で、一番大変だったことは何ですか？ どのように乗り越えましたか？

工作中的

① 状況

開発実習のプロジェクトに参加し、先輩と一緒にメタデータ加工クラスを実装していました。その際、メソッド名の重複（じゅうふく）によりコードが上書きされ、開発が滞る(とどこおる)リスクが生じ(しょうじ)ました。

② 課題

当時は実習生として知識や経験が不足しており、特にバージョン管理やコンフリクトへの対応方法に苦労していました。

③ 措施

そこで、まずGitの操作方法を調べ直し、先輩に状況を相談しながら、新しいブランチを作成し、クラス名や実装内容の一つずつ調整しました。必要に応じて複数回の試行を繰り返し、慎重に対応しました。

④ 成果

その結果、チームの開発進行に影響を与えることなく問題を解決でき、予定通りメタデータ加工クラスの実装を完了させ、先輩からも冷静に対応できたと評価されました。

⑤ 收获

この経験から、**冷静な状況分析とチームとの協調**の大切さを学び、今後も課題に直面した際は、自発的に行動し改善を試みる姿勢を大切にしていきたいと考えています。

試みる ころみる 尝试、试图、努力去做

協調 きょうちょう

生じる読み：しょうじる 罗马音：shōjiru

意味：发生、产生、出现

学生时代的

① 状況

日本語の勉強をしていた際、特に N1 レベルの学習において、語彙や文法の難しさ、長文読解のスピード不足に直面し、なかなか成績が上がらず、不安を感じる状況が続きました。

② 課題

このままでは試験に合格することが難しいと考え、学習方法を改善する必要がありました。

③ 措施

そこで、私は毎日の学習習慣を工夫しました。具体的には、毎日決まった時間に問題を解き、わからない部分は参考書で調べ、繰り返し復習にも積極的に取り組みました。

④ 成果

その結果、徐々に問題を解くスピードや正確さが上がるようになり、最終的には日本語能力試験 N1 に合格することができました。

⑤ 收获

この経験から、自分で計画を立てて継続して行動すれば目標を達成できることを実感しました。今後も諦めずに努力する姿勢を大切にしていきたいと考えています。

語彙（ごい /goi） 意思： 词汇、单词量

長文読解（ちょうぶんどっかい /chōbun dokkai） 意思： 长篇文章阅读理解

最終的（さいしゅうてき /saishūteki） 意思： 最终、最后

継続（けいぞく /keizoku） 意思： 继续、持续、坚持

将来的目标

常に学び続け、新しい技術やトレンドに敏感なエンジニアになりたいです。また、チームメンバーと協力し、プロジェクトの成功に貢献できるリーダーシップを持つエンジニアを目指しています。

用户中心项目

- 选用 MyBatis + MyBatis-Plus 进行数据访问层开发，继承了MyBatis Plus 提供的接口，从而复用对数据库的基本增删改查操作代码，不用手动编写重复的操作，大幅提高了开发效率。

MyBatis + MyBatis-Plus を選択してデータアクセス層（そう）の開発を行い、MyBatis Plus が提供するインターフェースを継承（けいしょう）することで、データベースに対する基本的な追加（ついか）・削除（さくじょ）・更新（こうしん）・参照（さんしょう）操作のコードを再利用し、重複（ちょうふく）する操作を手動（しゅどう）で記述（きじゅつ）する必要がなくなり、開発効率を大幅（おおはば）に向上（こうじょう）させます。

選択（せんたく）

層　そう　　sō

汎用的　はんようてき hanyōteki。通用的

継承　　けいしょう　keishō

独自　　どくじ　dokuji

向上　　こうじょう　kōjō

参照　　さんしょう（sanshou）　　查询（对应“查”）

「复用」核心是**直接用框架封装好的通用方法/代码**，不用重复写增删改查的基础逻辑，本质就是调用现成方法执行SQL

这里可能出现的问题：

MyBatis-Plus とは何ですか？ どのような役割がありますか？ MyBatis との違いは何ですか？ （什么是MyBatis-Plus?它有什么作用?它和MyBatis有哪些区别?）

MyBatis Plus を使用する前は、自身で SQL Mapper マッピングファイルと xml 文を作成する必要がありますが、MyBatis Plus を使用した後は、インターフェースを定義して BaseMapper インターフェースを継承し、その中に提供されている既存のメソッドを呼び出すだけで済みます。

（使用MyBatis Plus前，需要自己编写SQL Mapper映射文件和xml语句,使用MyBatis Plus后，定义接口继承BaseMapper 接口并调用其中提供的现成方法即可）

MyBatis 内部ではJDBC がカプセル化されており、これを使用すると、開発者は SQL 文そのものに注目するだけで、ドライバーのロード、コネクションの作成、statement の作成といった煩雑（はんざつ）なプロセスを処理する必要がなく、開発効率が向上します

（MyBatis内部封装了JDBC，使用它后，开发者只需要关注SQL语句本身，不需要处理加载驱动、创建连接、创建statement等繁杂的过程，提高开发效率。）

MyBatis-Plus は MyBatis に対して二次カプセル化を行ったものです。一連の API メソッドとコード生成器を提供することで、繰り返しの CRUD（作成、読み取り、更新、削除）操作をより簡便にし、手動で SQL 文を作成する必要がなくなり、それによって開発効率を大幅に向上させます。

（MyBatis-Plus是对MyBatis 进行了二次封装。通过提供一系列的API方法和代码生成器，使重复的CRUD（创建、读取、更新、删除）操作更加简单，无需手动编写SQL语句，从而大幅提高开发效率。）

二次 にじ niji “二次”
カプセル化 かぶせるか kapuseruka “封装”，
向上 こうじょう kōjō “提升、提高”，
効率：こうりつ (kōritsū)
生成器 せいせいき seiseiki
簡便 かんべん kanben
手動 しゅどう

- 明确接口的返回，自定义统一错误码：
インターフェースの返り値を明確にし、統一的なエラーコードをカスタマイズします。

カスタマイズ（かすたまいず） Customize 自定义
返り値 かえりち (ka e ri chi)
統一的 とういつてき (tō itsu teki)

我是通过一个枚举类(ErrorCode)来集中定义错误码 这个枚举类提供两个属性，code ， message
私は列挙型クラス（ErrorCode）を通じてエラーコードを集中的に定義しています。この列挙型クラスには二つのプロパティ（code、message）が備わっています。

code表示http的状态码 message 是返回给前端 给人看的简短说明文字 避免将后端真实的报错输出给前端 导致用户感知不好

code は HTTP ステータスコードを表し、message はフロントエンドに返却される、人が読むための簡潔な説明文です。これにより、バックエンドの実際のエラー内容がフロントエンドに出力され、ユーザーの体感が悪くなることを回避します。

列挙型クラス れっきょがたくらす 枚举类
プロパティ ぷろぱてい propertiy “属性”，外来语转假名
備わっています そなわっています sonawatte imasu “具备、拥有”
ステータスコード すてーたす こーど status code 状态码

表示 あらわす arawasu “表示、代表”

返却 へんきゃく henkyaku 返回、
簡潔 かんけつ
出力 しゅつりょく
回避 かいひ kaihi 避免、规避

- 封装全局异常处理器，规范异常返回、屏蔽项目冗余报错。
グローバル例外ハンドラーをカプセル化し、例外の返却を規格化し、プロジェクトの冗長なエラー出力を遮断する。

例外（れいがい） 例外（れいがい） = 日语 IT 里标准的「异常」
規格化（きかくか）
返却（へんきゃく）
冗長（じょうちょう）
遮断（しゃだん）
出力 しゅつりょく 输出

为什么后端需要全局异常处理器？
なぜバックエンドにはグローバル例外ハンドラーが必要なのか

有可能在开发的时候 controller有出现异常 service层也有出现异常 能否定义一个东西来捕获整个业务的异常
開発時にコントローラーで例外が発生したり、サービス層でも例外が発生したりすることがありますが、業務全体の例外をキャッチする何かを定義することはできますか

全局异常处理器能够捕获应用程序中抛出的异常, 并对这些异常进行统一的处理, 从而避免因未处理的异常导致系统崩溃或无法正常工作, 能够提高系统的稳定性。

グローバル例外ハンドラーは、アプリケーション内でスローされた例外をキャッチし、それらの例外に対して統一的处理を行うことができます。

これにより、未処理の例外が原因でシステムがクラッシュしたり正常に動作しなくなったりすることを防ぎ、システムの安定性を高めることができます

統一的（とういつてき）

未処理 みしょり 未処理、没有处理过的

クラッシュ crash 崩溃、闪退、宕机

你是如何实现全局异常处理器的？

グローバル例外ハンドラーはどのように実現するのですか？

定义一个全局异常处理类, 并且使用 @RestControllerAdvice (或者@ControllerAdvice) 注解该类, 表示这是一个全局异常处理器类。

グローバル例外処理クラスを定義し、@RestControllerAdvice（または@ControllerAdvice）アノテーションを付与することです。

アノテーション annotation 注解

付与（ふよ） 付与する = 添加

然后针对每种异常类定义一个处理异常的方法, 并且使用@ExceptionHandler (异常类, class) 注解标注这些方法, 可以在这些方法中进行日志记录等具体的异常处理操作, 并返回一个响应对象, 便于前端识别并给用户友好的错误提示。

その後、各種例外クラスに対して例外処理用のメソッドを定義し、@ExceptionHandler (例外クラス.class) アノテーションでこれらのメソッドを修飾します。これらのメソッド内でログ出力など具体的な例外処理を実装し、レスポンスオブジェクトを返却することで、フロントエンドが識別しやすく、ユーザーに対して親切なエラーメッセージを表示できるようになります。

各種（かくしゅ）

修飾（しゅうしょく）

出力 しゅつりょく 輸出

レスポンスオブジェクト Response Object 响应对象

返却（へんきゃく）

識別（しきべつ）

- 通过Spring Boot的多套 application-{env}.yml 配置文件实现多环境，并通过指定—spring.profiles.active=Prod 实现生产环境部署

Spring Boot の複数の application-{env}.yml 設定ファイルを通じて複数環境を実現し、--spring.profiles.active=Prodを指定することで本番環境へのデプロイを実現します

デプロイ 部署

主要修改的是 数据库的地址

主に修正したのはデータベースのアドレスです

登录如何实现

(这里到时候需要翻译成日文)

登录逻辑

1. 校验用户账户和密码是否合法
2. 非空
3. 账户长度不小于4位
4. 密码就不小于8位
5. 账户不包含特殊字符
6. 校验密码是否输入正确, 要和数据库中的密文密码去对比
7. 用户信息脱敏, 隐藏敏感信息, 防止数据库中的字段泄露
8. 返回脱敏后的用户信息

术语积累

1. 权限鉴定

- 権限認証 (けんげん にんしょう)
最自然、最常用。

2. 鉴权 (IT 术语)

- 認証 (にんしょう)
日文里鉴权 = 認証, 几乎所有文档都这么写。
- 更专业一点: アクセス認証 (访问鉴权)

トレンド torendo 趋势、潮流、热搜 对应英语 "Trend"

前后端分离 (IT 标准日语)

フロントエンドとバックエンドの分離 (読み: フロントエンド と バックエンド の ぶんり)

实习经历

浙江菜鸟サプライチェーン管理有限公司 (製品技術部 - 開発エンジニア JAVA)

2024 年 05 月 ~ 2024 年 09 月

実務背景

负责物流中钱和账相关内容, 追踪和处理物流链中各环节产生的费用数据, 简化物流计费结算流程

物流における金銭・会計関連業務を担当し、物流チェーン各段階で発生する費用データの追跡と処理を行い、物流の課金・決済プロセスの簡素化に従事。

负责物流中钱和账相关内容，追踪和处理物流链中各环节产生的费用数据，简化物流计费结算流程

浙江（せっこう）菜鸟（さいによ）

背景（はいけい）

課金（かきん） / kakkin

決済（けっさい） / kessai

簡素化（かんそか）

主な業務内容

プレビュー環境における異常トラフィックの監視が行われていない課題に対し、既存の監視ログのキー項目を基に、Sunfire 監視プラットフォームにてトラフィック監視ルールを設定。異常トラフィックのリアルタイムアラートを実現し、資産損失リスクの低減に貢献。

针对预发环境异常流量无监控的问题，基于现有监控日志关键字段配置，在Sunfire监控平台配置流量监控规则，实现异常流量实时告警，降低资损风险

既存（きぞん）

項目（こうもく）

低減（ていげん）

プレビュー preview 预览、预发、测试环境

トラフィック traffic 流量（网络流量、访问量）

韓国向け配送の課金ロジックを最適化するため、プロジェクト内でメタデータ加工処理を実装。localOpTimeStr 時間項目の解析を行い、書式を検証するとともに、韓国現地時間が日曜日かどうかを判定し、該当フラグ（Y/N）を返却する機能を開発。

为优化韩国配送计费逻辑，在项目中开发元数据加工方法，解析localOpTimeStr时间字段，校验格式并判断韩国当地时间是否为周日，返回对应标识（Y/N）

配送（はいそう）

課金（かきん） / kakkin 意思：课以费用 → 收取费用、计费

書式（しょしき）

検証（けんしょう）

該当フラグ 該当（がいとう） gaitō フラグ：flag 对应标识、对应标志位

返却

読み：へんきゃく

発音：henkyaku

意味：返回、返还、返回（値）

課金・決済チームにおける複数人での協調開発の流れを熟知し、Git を用いた日常的な開発運用を習得

熟知（じゅくち）

用いた（もちいた） mochiita 使用了、利用了

チームでの協調開発プロセスに従事し、要件定義に基づきブランチ開発、単体テスト、コードレビューを実施。コードカバレッジ基準の遵守、ログ出力規範の順守、コンフリクト回避、コードの堅牢性向上など、品質管理を徹底した。

参与团队协作开发流程，根据需求文档进行分支开发、单元测试、代码评审；严格遵守代码覆盖率标准、日志规范，避免代码冲突，注重代码健壮性与系统品质。

关键词

要件定義 → ブランチ開発 → 単体テスト → コードレビュー → 品質管理

従事じゅうじ /juuji从事、参与

遵守じゅんしゅ /junshu

ログ出力規範ろぐしゅつりょく きはん rogu shutsuryoku kihan 日志输出规范

順守じゅんしゅ /junshu 遵守、恪守（和遵守几乎同义）

堅牢性けんろうせい /kenrousei 健壮性、鲁棒性（IT 常用）

向上こうじょう /koujou 提升、提高